



Niklas Glaser

Heinrich-Heine-Universität Düsseldorf

April 2026



Overview

- 1 Motivation
- 2 Approach
- 3 Experiments
- 4 Results
- 5 Discussion
- 6 Conclusion

Goals and Research Question

Goal

- improve the Base Agent with League Training (LT)
- training against a more diverse set of opponents
- focus on robustness and generalization

Base Agent

- combination of Behavioral Cloning (BC) and Proximal Policy Optimization (PPO)
- strong baseline, but weaknesses against several opponents

Research Question

- How much does LT with PPO improve robustness and generalization of the BC/PPO-pretrained agent?
- How strongly does this depend on the diversity of initial agents and BC experts?

Setting

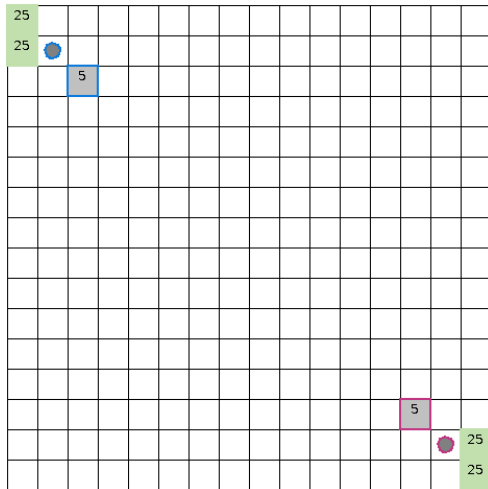
MicroRTS

- simplified RTS environment for RL research
- still contains resource management, unit production and combat
- supports controlled and repeatable experiments

Main RL challenges

- large combinatorial action space
- simultaneous decisions
- delayed and sparse rewards
- robustness against diverse opponents

► [Open MicroRTS Example Video](#)



Training Pipeline



BC and BC-FT

- imitate experts
- BC-FT: only winning replays

PPO

- iteratively update policy
- refine the policy through exploration

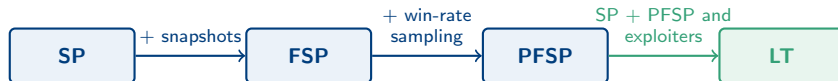
LT

- train against an evolving opponent population
- add exploiters

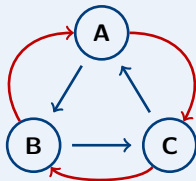
PPO fine-tuning

- repair the main remaining weakness after LT
- add WorkerRushAI as opponent

From SP to LT



cyclical learning of SP



Blue arrows: learning direction
Red arrows: policies the current policy performs well against

Why LT helps

- improves stability by identifying and exploiting weaknesses
- creates diverse set of opponents without the main agent having to find them through exploration

Main Agent (MA)

- primary policy
- resulting agent

Historical Agents (HA)

- frozen checkpoints kept in the opponent pool
- created by all other league agent types

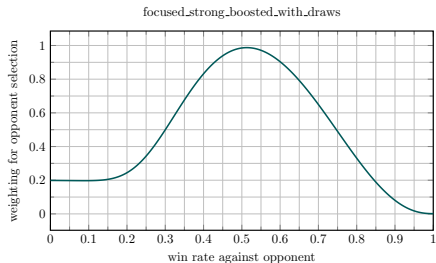
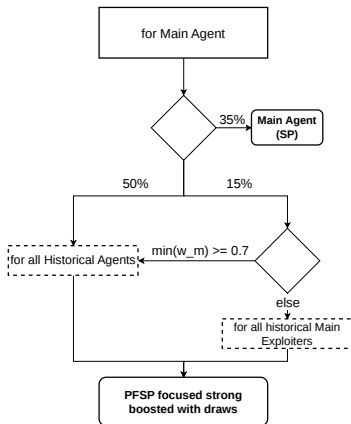
Main Exploiters (ME)

- exploit weaknesses of the MA
- reset when checkpointed

League Exploiters (LE)

- exploit broader weaknesses across the league
- can reset when checkpointed

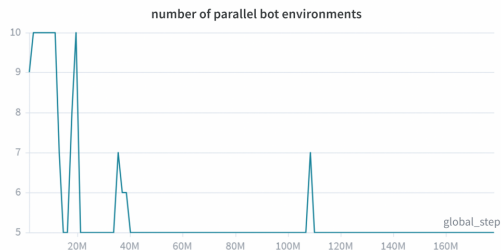
LT MA Matchmaking



- 35% self-play
- 50% PFSP over HAs
- 15% PFSP over HMEs if win rate is low
- PFSP prioritizes unresolved matchups

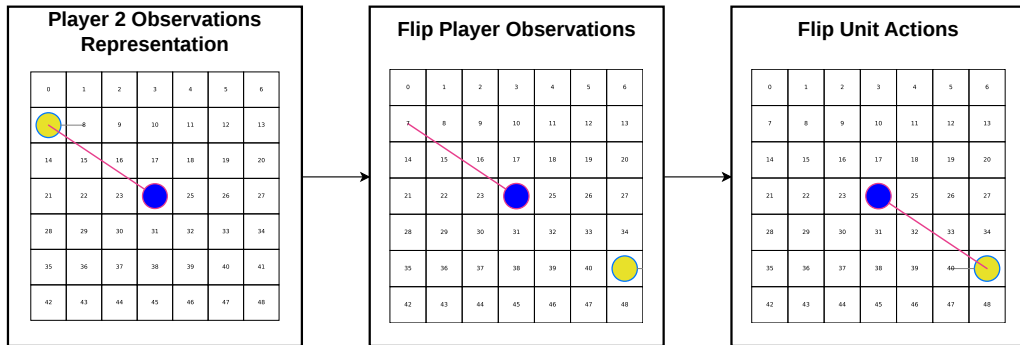
Bot Environments

- additional dynamic bot environments (CoacAI and Mayari)
- scripted bots: CoacAI, Mayari, WorkerRushAI, PassiveAI, LightRushAI, RandomAI and RandomBiasedAI
- MCTS based bots: Rojo, MixedBot, Izanagi, Tiamat, Droplet, GuidedRojoA3N and NaiveMCTS



Observation Adjustment for SP

- Base Agent only trained as Player 1
- flip Player-2 GridNet observations
- flip masks and action directions

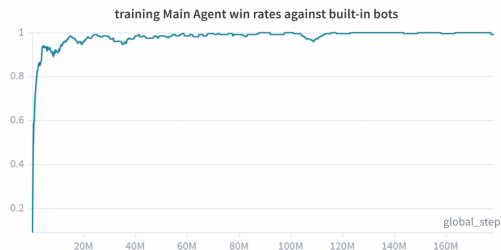


Implementation Changes

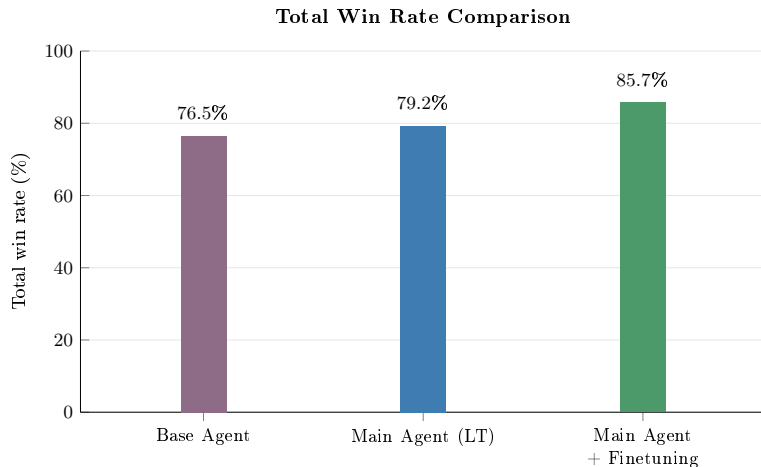
- delta score reward: denser signal in sparse-reward games
- annealed entropy coefficient: more exploration early, less later
- new initial agents: new start points for exploiters

Experiment Setup

- 1 map: basesWorkers16x16A
- 1 MA, 4 ME, 1 LE
- for each 25 environments
- 5 to 10 dynamic bot environments
- ~130M LT steps, ~100 h
- max: 160 parallel environments
- eval: at least 100 games vs 12 opponents



Main Quantitative Result



- loss rate: **7.7% → 2.2%**
- different Base Agent version than in the prior work

Evaluation Results (Win Rate)

Opponent	Base	LT	LT + FT
CoacAI	96.0	100.0	100.0
Mayari	67.3	100.0	100.0
WorkerRushAI (FT)	100.0	35.0	99.5
RandomBiasedAI	79.0	97.0	97.5
MixedBot	43.3	57.0	60.5
GuidedRojoA3N	74.7	87.0	88.5
Tiamat	93.0	99.0	99.0
Droplet	67.7	62.0	75.5
overall	76.5	79.2	85.7

- training bots: CoacAI and Mayari in LT, WorkerRushAI in FT
- RBAI: better against drawing bots
- WRAI: clear LT weakness, then FT training opponent
- LT gains stay after FT
- all MCTS win rates improve after FT

Evaluation Results (Loss Rate)

Opponent	Base	LT	LT + FT
WorkerRushAI (FT)	0.0	55.0	0.5
MixedBot	34.3	20.0	13.5
Izanagi	16.3	7.0	6.0
overall	7.7	7.5	2.2

- overall loss rate: less than **1/3** of the Base Agent
- better base defense while attacking

Interpretation of the Results

- robustness against diverse opponents becomes part of the optimization target
- LT finds a better local optimum
- FT improves this local optimum further
- MA after LT vs Base Agent: **89%** win, **9%** draw, **2%** loss

Player 1 (P1) Priority

- P1 is prioritized in conflicts
- self-match over 200 games: P1 **38%**, draw **28%**, P2 **34%**
- visible effect, but small
- alternating the learning side prevents P1-specific behavior

▶ [Open Example Video](#)

Main Limitation: Diversity

- not diverse exploiters discover weaknesses with similar exploits
- strategically different exploits require unlearning Base Agents fundamental behavior first
- long transitions, bad rewards
- new initial agents → more exploiter diversity

Hypothesis

- LT depends strongly on the diversity already present at initialization

► [Open Example Video](#)

Remaining Limitations and Future Work

Remaining limitations

- 1 map
- full observability, no fog of war
- high memory and computation cost
- 160 LT envs vs 24 for pure PPO

Future work

- more maps
- fog of war
- MicroRTS on GPU
- stronger diversity mechanisms

Conclusion

- LT + PPO improves robustness and generalization of the MA
- PPO fine-tuning further improves the final agent
- LT improves training and evaluation matchups
- effectiveness of LT depends on exploiter diversity
- exploiter diversity depends on initial agents and BC experts

Thank you

Questions?

Detailed Evaluation Results: Win Rates

Opponent	Main Agent without new initial agents	Base Agent	Main Agent (LT)	Main Agent + Fine-tuning
<i>bots used in training</i>				
CoacAI	100.0	96.0	100.0	100.0
Mayari	100.0	67.3	100.0	100.0
<i>scripted bots</i>				
WorkerRushAI	31.0	100.0	35.0	99.5
PassiveAI	100.0	99.7	100.0	100.0
LightRushAI	100.0	98.0	100.0	100.0
RandomAI	100.0	99.0	100.0	100.0
RandomBiasedAI	91.0	79.0	97.0	97.5
<i>MCTS based bots</i>				
Rojo	100.0	81.0	100.0	100.0
MixedBot	45.0	43.3	57.0	60.5
Izanagi	68.0	71.7	70.0	72.5
Tiamat	94.0	93.0	99.0	99.0
Droplet	45.0	67.7	62.0	75.5
GuidedRojoA3N	64.0	74.7	87.0	88.5
NaiveMCTS	0.0	0.7	2.0	6.5
overall	74.1	76.5	79.2	85.7

Detailed Evaluation Results: Loss Rates

Opponent	Main Agent without new initial agents	Base Agent	Main Agent (LT)	Main Agent + Fine-tuning
<i>bots used in training</i>				
CoacAI	0.0	2.0	0.0	0.0
Mayari	0.0	27.3	0.0	0.0
<i>scripted bots</i>				
WorkerRushAI	63.0	0.0	55.0	0.5
PassiveAI	0.0	0.0	0.0	0.0
LightRushAI	0.0	2.0	0.0	0.0
RandomAI	0.0	0.0	0.0	0.0
RandomBiasedAI	0.0	0.0	0.0	0.0
<i>MCTS based bots</i>				
Rojo	0.0	5.3	0.0	0.0
MixedBot	33.0	34.3	20.0	13.5
Izanagi	15.0	16.3	7.0	6.0
Tiamat	6.0	7.0	1.0	0.5
Droplet	22.0	9.0	22.0	10.5
GuidedRojoA3N	0.0	1.7	0.0	0.0
NaiveMCTS	0.0	2.7	0.0	0.0
overall	9.9	7.7	7.5	2.2

Main Agent vs League Exploiter

- the League Exploiter resets
- different hyperparameters
- historical League Exploiters are not included as Main Exploiter opponents

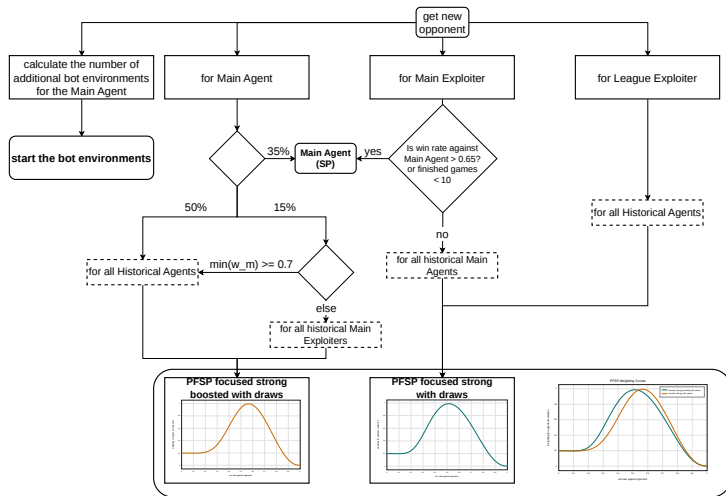
Why adjust the observations for SP?

- Gym-MicroRTS: Toward affordable full game real-time strategy games research with deep reinforcement learning implemented SP and concluded that a likely reason for worse performance of SP was that SP also has to train, to start as Player 2, while pure PPO only trained as Player 1.
- difficulties retrainig the Base Agent.

Why are the win rates against NaiveMCTS low?

- It is one of the few opponents that often gives up on winning and instead stays far away from the agent's units.
- This results in many draws and few losses.

Full LT Matchmaking



Matchmaking and opponent sampling in the LT setup.

New Initial Agents

1 agent per unit type

Heavy focused

- Base Agent

Light focused

- 9 epochs of BC with LightRushAI as the expert
- opponents for BC: Mayari, WorkerRushAI, LightRushAI

Worker focused

- 8 epochs of BC with WorkerRushAI as the expert
- opponents for BC: Mayari, WorkerRushAI, LightRushAI

Ranged focused

- 6 epochs of BC with CoacAI as the expert
- 180 updates of PPO against the Base Agent, CoacAI and Mayari
- opponents for BC: WorkerRushAI, LightRushAI, Mayari and CoacAI

Why Not Fewer Environments?

- The number of environments affects rollout size.
- Smaller rollouts make PPO updates more unstable because they provide less information about the update direction.
- More steps per rollout create similar tradeoffs to adding more environments.